POLITÉCNICA
"Ingeniamos el futuro"

CAMPUS
DE EXCELENCIA
INTERNACIONAL

Front-end para Móviles (FEM)

Tema 4 - Conectividad



miw.etsisi.upm.es

F. Javier Gil

Tema 4: Conectividad

- 4.1 Consulta estado conectividad
- 4.2 Conexión a recursos remotos
  - Clase HttpURLConection
- 4.3 Resolución de Tareas
  - AsyncTask, WorkManager
- 4.4 Conexión recursos y servicios web
  - Picasso, Glide, Retrofit, Volley, etc.
- 4.5 Autenticación y almacenamiento de datos en la nube
  - Firebase Authentication, Realtime Database, Cloud Storage

Consulta estado conectividad

- Permisos:
  - `<uses-permission android:name="android.permission.INTERNET" />`
  - `<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />`
- Comprobación del estado:

```
public boolean hayConexion() {
    ConnectivityManager conMgr = (ConnectivityManager)
                getSystemService(Context.CONNECTIVITY_SERVICE);
    Network nw = conMgr.getActiveNetwork();
    if (nw == null) return false;
    NetworkCapabilities actNw = conMgr.getNetworkCapabilities(nw);

    return (actNw != null &&
     actNw.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET));
}
```

- Problemas ANR → AsyncTask, Retrofit, Volley, etc.

Tema 4: Conectividad

- 4.1 Consulta estado conectividad
- 4.2 Conexión a recursos remotos
  - Clase HttpURLConection
- 4.3 Resolución de Tareas
  - AsyncTask, WorkManager
- 4.4 Conexión recursos y servicios web
  - Picasso, Glide, Retrofit, Volley, etc.
- 4.5 Autenticación y almacenamiento de datos en la nube
  - Firebase Authentication, Realtime Database, Cloud Storage

Clase HttpURLConection (I)

- Cliente HttpURLConection:

```
HttpURLConnection con = null;
try {
    URL mUrl = new URL(urlRecurso);
    con = (HttpURLConnection) mUrl.openConnection();
    InputStream is = con.getInputStream();
    // @TODO obtener resultado...
    is.close();
} catch (Exception e) {
    // @TODO tratamiento error
} finally {
    if (con != null) con.disconnect();
}
```

https://developer.android.com/reference/java/net/HttpURLConnection

Clase HttpURLConection (II)

- HttpURLConection principales métodos
  - `get / setRequestMethod(String method)`
    - método: GET | POST | HEAD | OPTIONS | PUT | DELETE | TRACE
  - `getHeaderField(int n)`, `getHeaderFieldKey(int n)`, ...
    - acceso a las cabeceras de los mensajes de respuesta
  - `getResponseCode()`, `getResponseMessage()`
    - código y cuerpo del mensaje de respuesta
  - `get / setFollowRedirects(boolean set)`
    - redireccionamiento automático
  - `disconnect()`
    - Indica que no se van a realizar más peticiones próximamente
  - `getErrorStream()`
    - devuelve el error en caso de fallo de la conexión

https://developer.android.com/reference/java/net/HttpURLConnection

Tema 4: Conectividad

- 4.1 Consulta estado conectividad
- 4.2 Conexión a recursos remotos
  - Clase HttpURLConection
- 4.3 Resolución de Tareas
  - AsyncTask, WorkManager
- 4.4 Conexión recursos y servicios web
  - Picasso, Glide, Retrofit, Volley, etc.
- 4.5 Autenticación y almacenamiento de datos en la nube
  - Firebase Authentication, Realtime Database, Cloud Storage

Tareas asíncronas: AsyncTask (I)

- ejecuta tareas en background y muestra resultados en la interfaz de usuario
- diseñada para operaciones cortas (pocos segundos)
- definida por tres tipos genéricos:
  - `Params`: tipo de los parámetros enviados a la tarea para su ejecución
  - `Progress`: tipo de las unidades para mostrar el progreso
  - `Result`: tipo devuelto por la tarea ejecutada en background
- no es obligatorio indicar todos los tipos
  - tipo no utilizados: especificar `Void`

```
private class MiTarea extends AsyncTask<Void, Void, Void>
```

https://developer.android.com/reference/android/os/AsyncTask

Tareas asíncronas: AsyncTask (II)

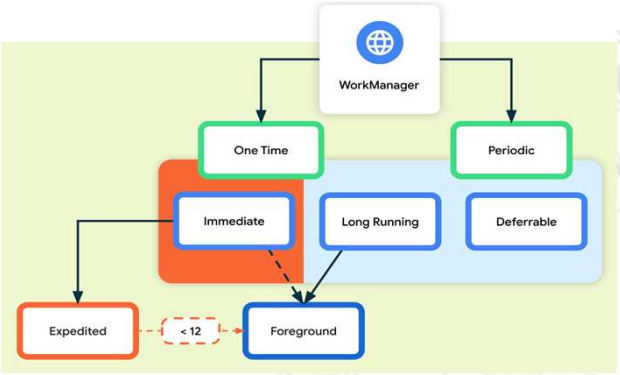
- métodos:
  - **onPreExecute()**: inicialización, ajustes, etc.
  - **doInBackground(**Params... params**)**
    - ejecuta la tarea en background
    - puede emplear el método publishProgress(Progress...) para mostrar el progreso
    - genera un objeto de la clase Result
  - **onProgressUpdate(**Progress... progress**)**: muestra el progreso de la tarea
  - **onCancelled(**Result result**)**: se ejecuta cuando se cancela la tarea. Evita la ejecución de onPostExecute()
  - **onPostExecute(**Result result**)**: recibe el resultado de la tarea en background

Tareas asíncronas: AsyncTask (III)

- métodos públicos:
  - **execute(**Params... params**)**
    - ejecuta la tarea
  - **cancel(**boolean mayInterruptIfRunning**)**
    - intenta cancelar la ejecución de la tarea
  - **getStatus()**
    - devuelve el estado de la tarea
    - AsyncTask.Status: PENDING | RUNNING | FINISHED
  - **get(**long timeout, TimeUnit unit**)**
    - espera durante el tiempo especificado como parámetro y después devuelve el resultado.
  - **isCancelled()**
    - determina si la tarea ha sido cancelada

Programación de tareas: WorkManager

- API principal recomendada para el procesamiento en segundo plano
- recomendada para tareas persistentes
  - permanecen programado a través de reinicios de la app y del sistema
- tipos de tarea:



Programación de tareas: WorkManager

- definición del objetivo de la tarea en background a través de la clase `Worker`
  - implementar método `doWork()` que recibe `inputData` y devuelve objeto `Result`
- programación del trabajo: objeto `WorkRequest`
  - define cómo y cuándo se debe ejecutar la tarea
  - especificación restricciones: clase `Constraint`
  - permite encadenar trabajos secuenciales o en paralelo
- encolar la petición: método `enqueue()` del `WorkManager`

`https://developer.android.com/topic/libraries/architecture/workmanager`

Tema 4: Conectividad

- 4.1 Consulta estado conectividad
- 4.2 Conexión a recursos remotos
  - Clase HttpURLConection
- 4.3 Resolución de Tareas
  - AsyncTask, WorkManager
- 4.4 Conexión recursos y servicios web
  - Picasso, Glide, Retrofit, Volley, etc.
- 4.5 Autenticación y almacenamiento de datos en la nube
  - Firebase Authentication, Realtime Database, Cloud Storage

Conexión recursos web: Picasso

- descarga de imágenes, recursos, ficheros y librería caché
- transformación de imágenes con consumo mínimo de memoria
- caching automático de memoria y disco

```
Picasso.get()
      .load(url)
      .resize(50, 50)
      .centerCrop()
      .into(imageView)
```

http://square.github.io/picasso/

Conexión recursos web: Glide

- gestión de medios y carga de **imágenes**
- soporta GIF animados
- capacidad de generación de thumbnails
- formato de bitmap por defecto RGB_565

```
Glide.with(context, activity, fragment, …)
    .load(url)
    .override(300, 200)
    .centerCrop()
    .error(R.drawable.placeholder)
    .into(imageView)
```

https://bumptech.github.io/glide/

Conexión servicios web: Retrofit

- cliente REST para Android y Java con soporte HTTP/2
- realiza peticiones HTTP síncronas o asíncronas (GET, POST, PUT, PATCH, DELETE y HEAD) y parsea a POJOs
- convierte servicio web en una **interfaz** Java
  - implementación POJOs
  - especificación interfaz API REST

```
public interface CountryRESTAPIService {

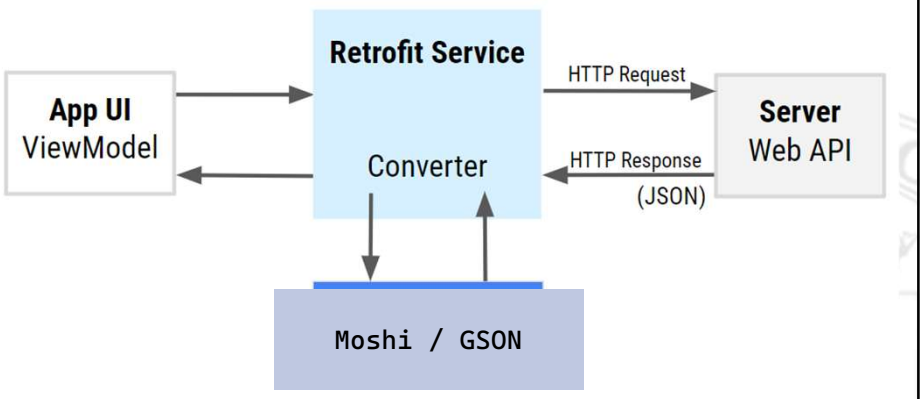
    // https://restcountries.com/v3.1/name/España
    @GET("v3.1/name/{countryName}")
    Call<List<Country>> getCountryByName(@Path("countryName") String countryName);
}
```

  - uso de retrofit

https://square.github.io/retrofit/

Conexión servicios web: Retrofit

- End-to-end diagram



Conexión servicios web: Volley

- librería para la gestión de peticiones HTTP
- procesamiento priorizado y concurrente
- caché de disco y memoria
- basado en gestión de una cola –única- de peticiones
- optimización de descarga de imágenes (`ImageLoader`)

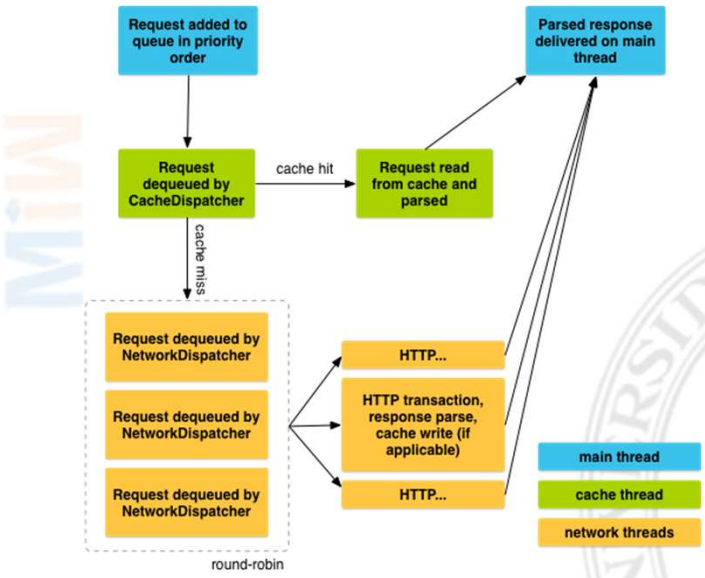
1. añadir permiso (`android.permission.INTERNET`)
2. incluir dependencias en gradle:

```
dependencies {
    ...
    implementation 'com.android.volley:volley:<versionVolley>'
}
```

3. creación de la cola de peticiones (patrón `Singleton`)
4. encolar peticiones (tipo `StringRequest`, `JsonObjectRequest`, `JsonArrayRequest`, `ImageRequest`)

`https://developer.android.com/training/volley`

Conexión servicios web: Volley

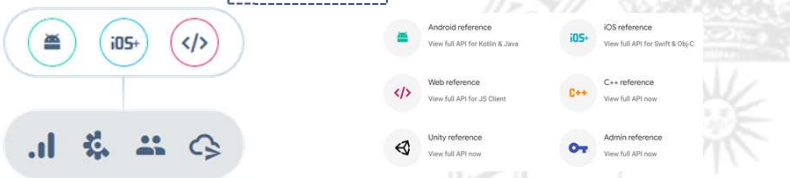


Tema 4: Conectividad

- 4.1 Consulta estado conectividad
- 4.2 Conexión a recursos remotos
  - Clase HttpURLConection
- 4.3 Resolución de Tareas
  - AsyncTask, WorkManager
- 4.4 Conexión recursos y servicios web
  - Picasso, Glide, Retrofit, Volley, etc.
- 4.5 Autenticación y almacenamiento de datos en la nube
  - Firebase Authentication, Realtime Database, Cloud Storage

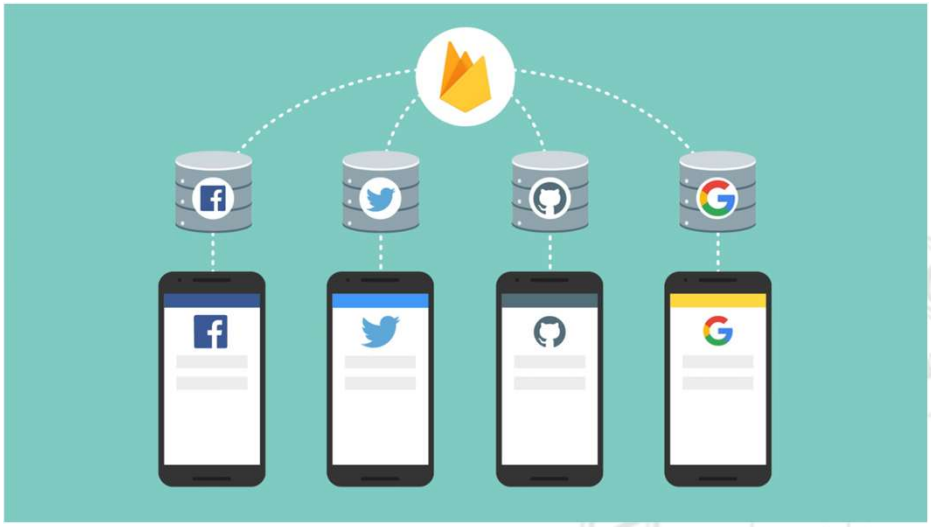
Añadir Firebase a un proyecto

- asistente Android Studio: `Tools > Firebase`
- proceso manual:
  - crear proyecto en consola de Firebase
    - https://console.firebase.google.com/
  - asociar Firebase con aplicación (`google-services.json`)
    - especificar el nombre del package
    - obtener la clave SHA1 del almacén de claves
    - añadir bibliotecas (Android BoM) y plugin google-services



`https://firebase.google.com/docs/android/setup`

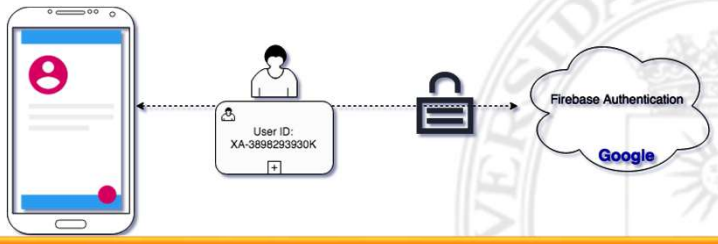
Autenticación con Firebase Authentication



https://firebase.google.com/docs/auth/

Autenticación con Firebase Authentication

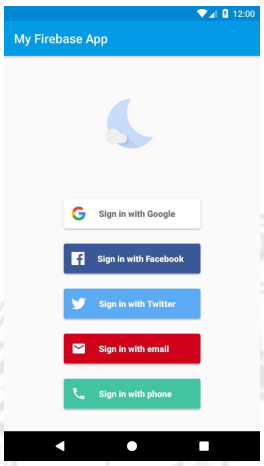
- facilita la autenticación a usuarios y desarrolladores
- compatible con Android, iOs y Web
- necesario para proveer navegación personalizada
- built in para Facebook, Twitter, Github, Gmail, em@il
  - User ID: XA-3898293930K único y común a todos ellos
- interfaz de usuario Open Source personalizable



https://firebase.google.com/docs/auth/

Autenticación con FirebaseUI

- proporciona flujos directos de IU para integrar en las apps
  - múltiples proveedores
  - administración de cuentas
  - vinculación de cuentas
  - actualización usuarios anónimos
  - temas personalizados
  - integración con Smart Lock (acceso desde múltiples dispositivos)



https://firebase.google.com/docs/auth/android/firebaseui

Persistencia con Firebase Realtime Database

- sistema NoSQL de almacenamiento multiusuario
- sincronización en tiempo real entre dispositivos
- adaptado a sincronización online/offline
- datos almacenados en estructura JSON
  - Primary Keys: Push Ids generados por Firebase



https://firebase.google.com/docs/database/

Persistencia con Firebase Realtime Database

- creación de una base de datos:
  1. crear la base de datos en Firebase desde la consola
  2. elegir modo inicial de reglas de seguridad:
     - modo prueba: acceso temporal, sin restricciones
     - modo bloqueado: rechaza todas las operaciones
  3. seleccionar ubicación de la instancia
  4. desplegar la base de datos
- conexión de la app:
  1. añadir dependencias (empleando BoM)
  2. obtener la instancia: `FirebaseDatabase.getInstance()`
  3. realizar operaciones de lectura / escritura

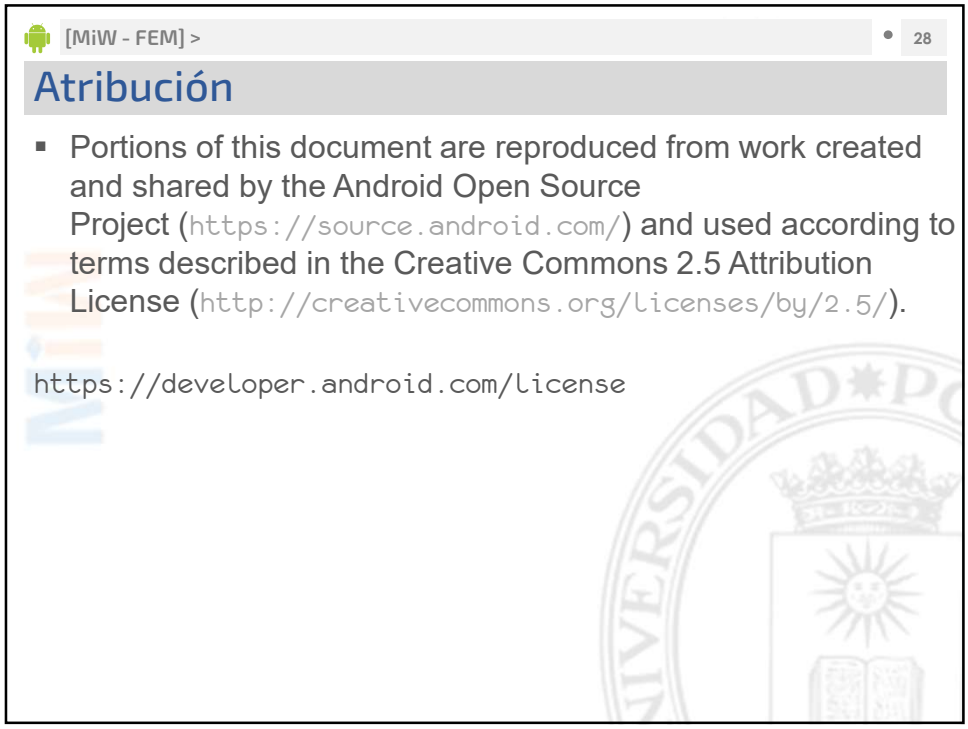
https://firebase.google.com/docs/database/android/read-and-write

Persistencia con Cloud Storage for Firebase

- implementa alojamiento de ficheros mayores como pueden ser imágenes, audio o vídeo
- permite carga/descarga adaptada al ancho de banda de cada usuario
- permite hacer restricciones de seguridad para indicar quién puede acceder a qué ficheros
- sobre conexión segura y rollback en caso de falllo



https://firebase.google.com/docs/storage/


[MiW - FEM] >
28
Atribución
Portions of this document are reproduced from work created and shared by the Android Open Source Project (https://source.android.com/) and used according to terms described in the Creative Commons 2.5 Attribution License (http://creativecommons.org/licenses/by/2.5/).
https://developer.android.com/license